

ICT2217

Network Security

Lab 1.2 Notes:

Internal Attacks over LAN and Switch Security Configuration I



Lab Exercise 1.2

1

Switch spoofing attack

2

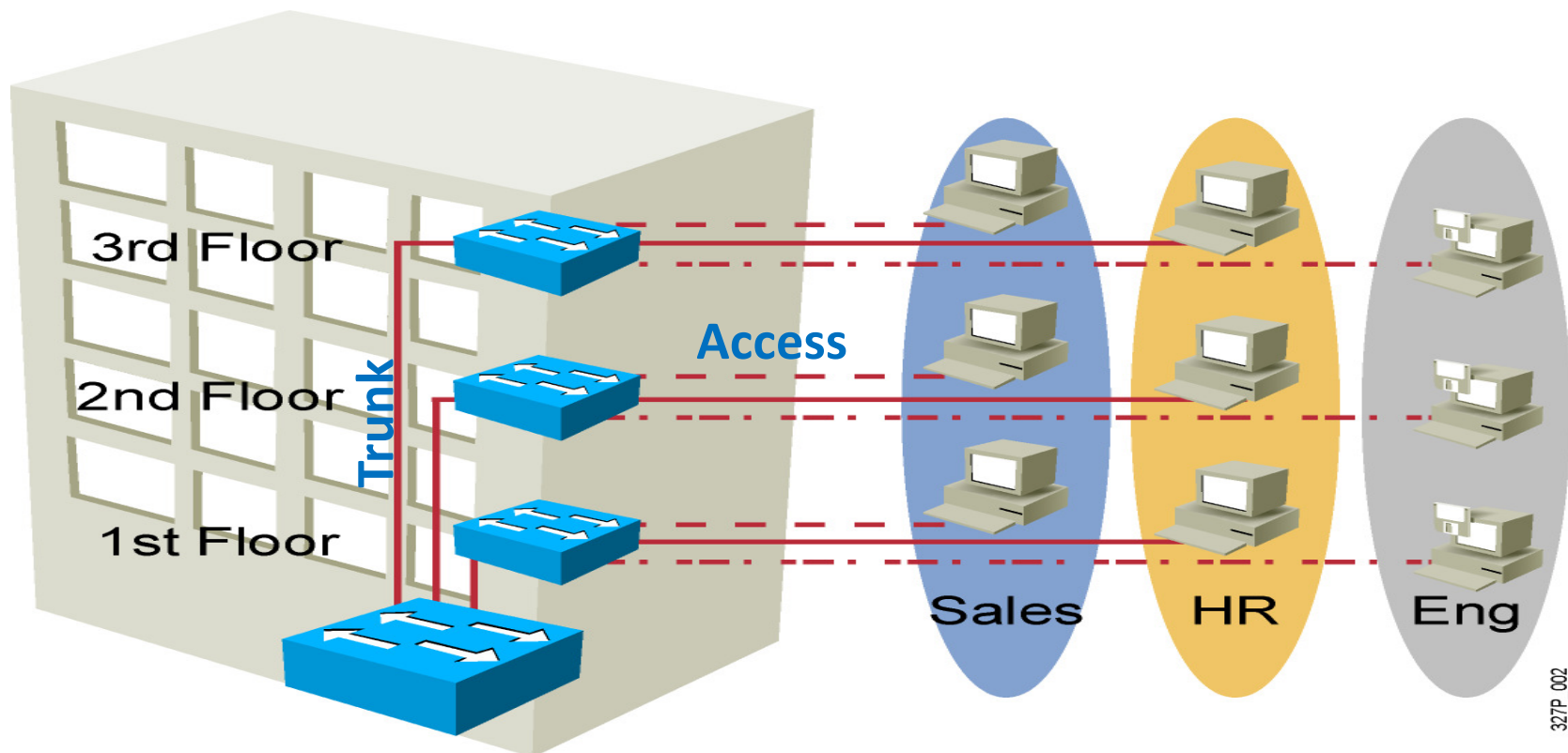
Defenses:

- Manually configure ports as access or trunk
- Disable DTP
- Good practice to shut down all unused ports and configure them into an unused VLAN

In ICT1013, you have learned about configuring **VLANs** (virtual LANs) on **switches** to separate frames in different VLANs to support users in different departments.

With VLANs, switch port connected to a link is operating either as:

- (i) **Access** – carry frames for **only one VLAN** configured on the port; or
- (ii) **Trunk** – carry frames for **multiple VLANs** on one physical link



To simplify configuration, modern switches are commonly shipped with mechanisms or protocols to automate trunk formation; e.g. the Cisco **Dynamic Trunking Protocol (DTP)**.

In Cisco switches, each **switch port** can be configured in 4 different modes: **access**, **dynamic auto**, **trunk** and **dynamic desirable**.

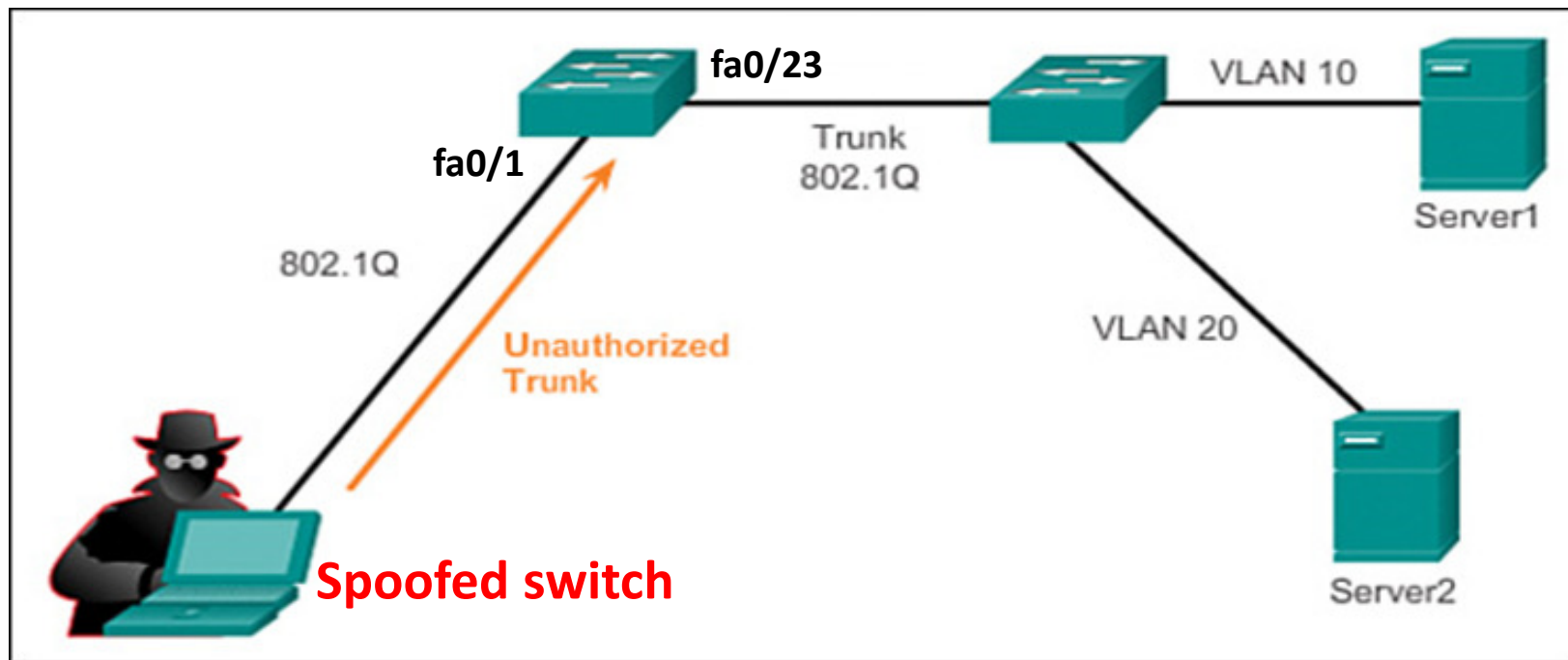
Then, DTP will be run automatically which will result in the **link** being an **access** or **trunk** as follows:

Administrative Mode	Access	Dynamic Auto	Trunk	Dynamic Desirable
access	Access	Access	Do Not Use ¹	Access
dynamic auto	Access	Access	Trunk	Trunk
trunk	Do Not Use ¹	Trunk	Trunk	Trunk
dynamic desirable	Access	Trunk	Trunk	Trunk

By **default**, Cisco switch ports are in **dynamic auto** mode which enables DTP and will passively wait to receive trunk negotiation to become a trunk.

Unfortunately, the use of DTP can lead to **switch spoofing attack** – a form of **VLAN hopping attack** where attackers could access frames in other VLANs that normally would not be accessible.

Taking advantage of the default configuration of the switch port which is dynamic auto, an attacker can inject DTP messages to form a trunk.



With the **spoofed switch**, the attacker may now **receive frames** from other VLANs, or **send spoofed frames** tagged for any target VLAN.

To demonstrate **switch spoofing attack**, you may use the tool **Yersinia**, which is available in Kali Linux.

== Launching Yersinia in interactive mode:

```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ sudo yersinia -I
```

```
kali@kali: /opt/yersinia  
File Actions Edit View Help  
yersinia 0.8.2 Available commands [21:45:59]  
RootId  
h Help screen  
x eXecute attack  
i edit Interfaces  
ENTER information about selected item  
v View hex packet dump  
d load protocol Default values  
e Edit packet fields  
f list capture Files  
s Save packets from protocol  
S Save packets from all protocols  
L Learn packet from network  
M set Mac spoofing on/off  
l List running attacks  
K Kill all running attacks  
c Clear current protocol stats  
C Clear all protocols stats  
g Go to other protocol screen  
Ctrl-L redraw screen  
w Write configuration file  
a About this proggie  
q Quit (bring da noise)  
Total Packets: 0  
This is the help s  
STP Fields  
Source MAC 0A:23:  
Id 0000 Ver 00 Ty  
BridgeId CB09.E7C  
MAC Spoofing [X]  
:00  
thcost 00000000  
o 0002 Fwd 000F
```

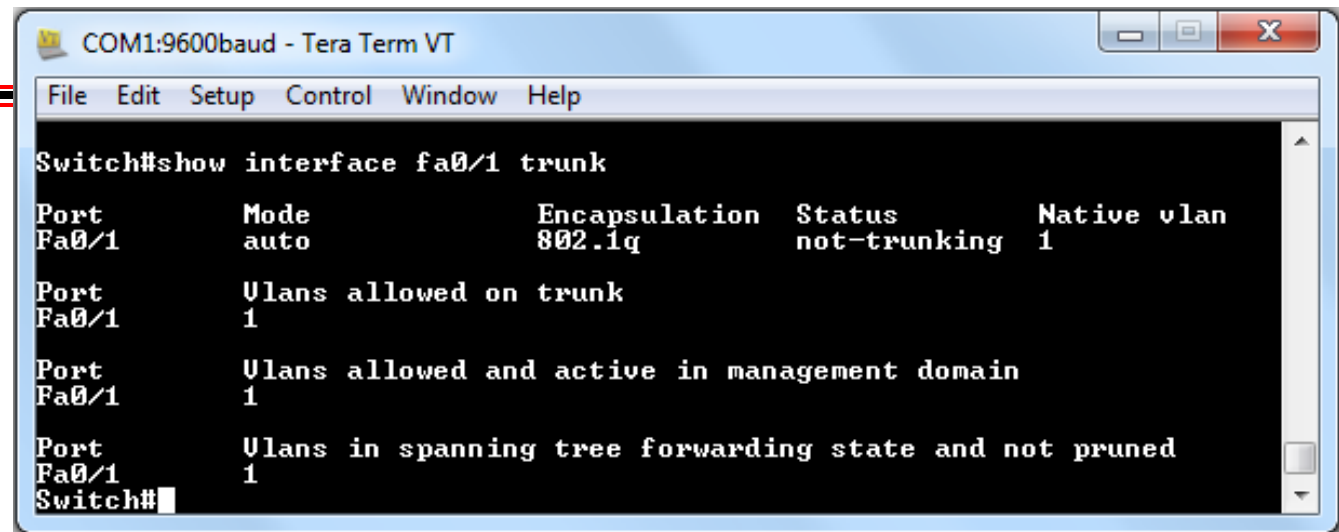
Press key 'h' to display help screen on the functionality of different keys.

```
kali@kali: ~  
File Actions Edit View Help  
yersinia 0.8.2 by Slay & tomac - STP mode [21:27:57]  
RootId BridgeId Port Iface Last seen  
Choose protocol mode  
CDP Cisco Discovery Protocol  
DHCP Dynamic Host Configuration Protocol  
802.1Q IEEE 802.1Q  
802.1X IEEE 802.1X  
DTP Dynamic Trunking Protocol  
HSRP Hot Standby Router Protocol  
ISL Inter-Switch Link Protocol  
MPLS MultiProtocol Label Switching  
STP Spanning Tree Protocol  
VTP VLAN Trunking Protocol  
ENTER to select - ESC/Q to quit  
Total Packets: 0 STP Packets: 0 MAC Spoofing [X]  
Choose your life (mode)  
STP Fields  
Source MAC 0A:23:16:02:FF:08 Destination MAC 01:80:C2:00:00:00  
Id 0000 Ver 00 Type 00 Flags 00 RootId 5080.760F0E14AC58 Pathcost 00000000  
BridgeId CB09.E7CD90117CAA Port 8002 Age 0000 Max 0014 Hello 0002 Fwd 000F
```

Press key 'g' to display protocol screen, arrow keys to navigate and ENTER key to select.

An example of the result of conducting **switch spoofing attack** on a switch using **Yersinia**.

Port Fa0/1 in
default
dynamic auto
before attack:



```
COM1:9600baud - Tera Term VT
File Edit Setup Control Window Help

Switch#show interface fa0/1 trunk

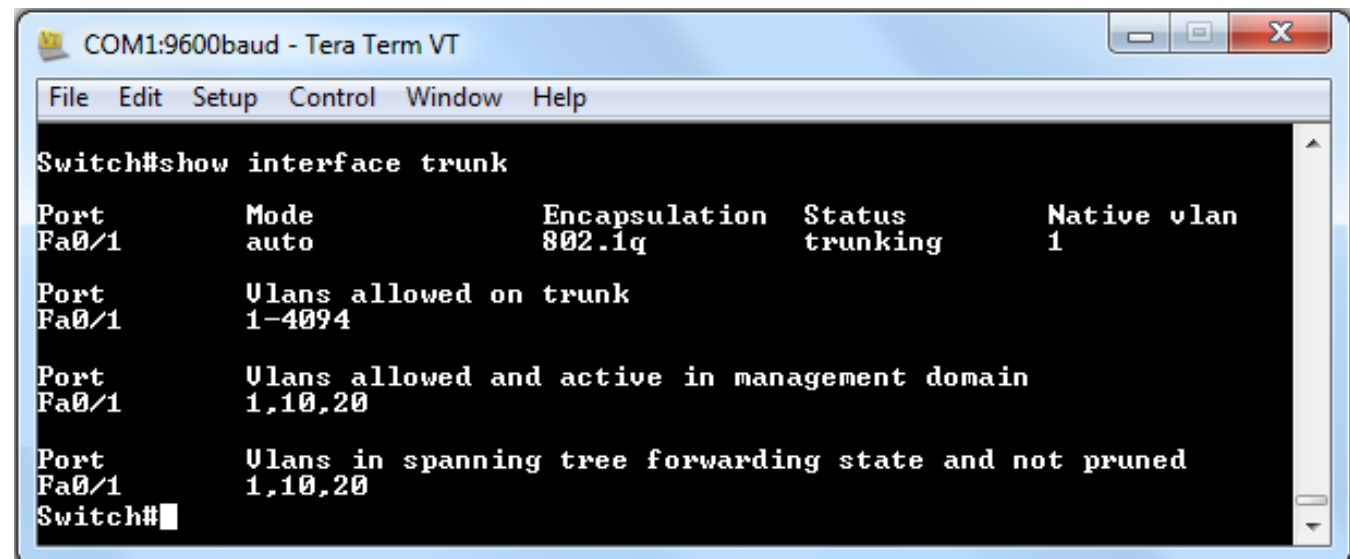
Port      Mode      Encapsulation  Status      Native vlan
Fa0/1     auto      802.1q         not-trunking 1

Port      Vlans allowed on trunk
Fa0/1     1

Port      Vlans allowed and active in management domain
Fa0/1     1

Port      Vlans in spanning tree forwarding state and not pruned
Fa0/1     1
Switch#
```

Port Fa0/1
becomes trunk
after attack:



```
COM1:9600baud - Tera Term VT
File Edit Setup Control Window Help

Switch#show interface trunk

Port      Mode      Encapsulation  Status      Native vlan
Fa0/1     auto      802.1q         trunking     1

Port      Vlans allowed on trunk
Fa0/1     1-4094

Port      Vlans allowed and active in management domain
Fa0/1     1,10,20

Port      Vlans in spanning tree forwarding state and not pruned
Fa0/1     1,10,20
Switch#
```


To prevent switch spoofing attack, disable DTP negotiations and manually configure ports as access or trunk explicitly.



```
SW2(config)# interface fa0/1
! Specifies that this is an access port
SW2(config-if)# switchport mode access
SW2(config-if)# switchport access VLAN 10
! Note, even for an interface configured as an access port,
! negotiation packets are still being sent unless we disable it
SW2(config-if)# switchport nonegotiate

SW2(config-if)# interface fa0/23
! Specifies the port as a trunk
SW2(config-if)# switchport mode trunk
SW2(config-if)# switchport nonegotiate
```


For added security, it is also wise to **shutdown all unused ports** and place them in an **unused VLAN** , e.g. 100, so as to prevent someone from discovering a live port that might be exploited.

For example, if **port Fa0/2-22** are **unused**, then **shut them down** to **prevent attacker** from discovering and exploiting.

